Aggregation Rules To Protect Against Poisoning Attacks In the Cronus Architecture's Black Box Collaborative Learning

Imad Khan*
iaman@umass.edu
University of Massachusetts Amherst
Amherst, Massachusetts, USA

Alexander Prentiss*
aprentiss@umass.edu
University of Massachusetts Amherst
Amherst, Massachusetts, USA

1 Introduction and Problem Statement

Collaborative learning is a mechanism to train machine learning models using multiple distributed clients that train with their own private data, while sharing information about their own local models to aid in training. Cronus is a black-box collaborative learning architecture introduced in [2] designed to improve security in collaborative learning from inference attacks, where attackers use shared information about the model to reconstruct the private training data of other models, and poisoning attacks, where attackers inject models with false information through information sharing to increase error rates.

The Cronus algorithm utilizes a directional filtering aggregation rule (AGR) called RobustFilter to rule out client updates that don't match the covariance of the majority of models. RobustFilter filters out data too far along each corrupted direction of the data and then takes a sample mean. This aggregation rule itself contributes to the architecture's power against various poisoning attacks, although it is unclear whether it is the best AGR within the Cronus system.

1.1 Our Contribution

Since other AGRs may be utilized within Cronus, as implied by the authors themselves, we investigate those that could be more robust to poisoning. In this paper we tested two alternatives to RobustFilter, Trimmed Mean and Geometric Median, as well as combinations of them with RobustFilter, against two attacks, Little Is Enough (LIE) and Label Flip, to explore ways to optimize the Cronus collaborative architecture. We used the MNIST dataset with convolutional neural network clients in the process.

2 Background

2.1 Cronus Architecture

The Cronus architecture paper [2] describes Cronus as a black-box collaborative learning model that, instead of traditional collaborative learning models, uses aggregated label predictions in order to train models collaboratively. In collaborative learning, client models train on their own in a private training phase, then share information about themselves in a collaborative phase, where a server aggregates this shared information to then share with client models. In federated learning models, the traditional approach that Cronus improves upon, homogenous models train on their own private training data. They then share their model parameters with a server, which aggregates them and then shares them back with participants. The models then update themselves with the aggregated parameters and then continue with training on their private data. This process is repeated until convergence.

*Both authors contributed equally to this research.

The Cronus paper raises many different issues with the federated learning model. First of all, they utilize homogenous models, meaning every model has to have the same architecture. For example, you cannot do collaborative learning with a Convolutional Neural Network with 5 hidden layers and another one with 4 hidden layers, as they would have different model parameters that couldn't be aggregated properly. Secondly, they are not robust against poisoning, where a malicious model participating in the collaborative learning process can send false parameter updates that cannot be filtered properly from the aggregation, thus slowly poisoning the models. Lastly, private data can be leaked through the model parameters, as parameter sharing in collaborative learning is essentially a white-box process that allows reverse-engineering of the private training data that each model used, posing serious security concerns. Inference attacks are what these are called, and the Cronus paper describes the methods through which these attacks occur.

Cronus alleviates all of these issues, through a collaborative learning environment that does public label distillation instead of parameter distillation. Public predictions on a public dataset are done by each of the clients with the server each collaboration round, then aggregated with an AGR, then shared back with the client models, who then train on their private data as well as these aggregated prediction data labels. This way, knowledge between the models is shared in a black-box manner. This robust method improves upon all the mentioned caveats with federated learning. It allows for heterogenous client models due to not sharing model parameters themselves, but only prediction labels. It enables additional robustness against poisoning, as instead of overwriting their parameters with an aggregate, models are instead fine-tuned (trained) with aggregated labels, reducing the likelihood of harmful updates having a significant impact. More information on the exact Cronus algorithm can be found in the paper [2].

2.2 Aggregation Rules (AGRs)

Aggregation rules (AGRs) enable a way of combining information between models shared between each other. In federated learning, an AGR is used to combine parameter updates sent to the server, which are then subsequently shared with clients. In Cronus, AGRs are used on the label predictions on the public dataset themselves, which are then shared with the client models for fine-tuning.

The Cronus paper uses a robust AGR called RobustFilter for its aggregation. The RobustFilter algorithm essentially computes a corrupted direction of the empirical mean of shared label data. With this computed direction, it filters out a random amount of the data farthest from the mean along that direction. It continues doing this until variance is minimized among every direction. This algorithm

is shown in detail in [2], and is what we use for our adaptation of the model for our experiment.

The Cronus paper compares the Cronus implementation with this AGR with federated learning models with various different AGRs. They compare against federated learning with a couple of different AGRs. They compare a geometric median scheme called Krum aggregation, as well as one called Bulyan, which combines Krum with a coordinate-wise filtering scheme called Trimmed-Mean. They also compare with an aggregation scheme called Multiplicative Weight Update (MWU), that uses the distances between malicious updates and aggregated updates. Of course, the Cronus algorithm with RobustFilter is shown as better than federated learning with any of these AGRs in the paper.

2.3 Model Attacks

There were many different collaborative learning attacks tested in the Cronus paper, but we will be focusing on two: Little is Enough (LIE) and Label Flip. The LIE attack, as described in [1], listens to server updates and provides it's own benign updates until and activation round. It uses these samples to craft an estimated model of updates, giving the model the ability to craft the maximum undetectable shift in updates in the same direction as the benign models, slowly poisoning the federated server over time. The other attack, label flip, is more overt. The models flips the labels of its own private training data to a specific target class. This shifts the median label of a data in the public set to be the target class. All clients do the same transformation to the labels, systemically poisoning the training data. Over time the rest of the benign models also learn to bias towards the target class.

3 Motivation

What the Cronus paper did not seem to address was the use of different aggregation rules within the architecture itself. Cronus compared using the RobustFilter AGR to aggregate labels in the architecture with different AGRs within a federated learning environment. However, it did not compare RobustFilter with other AGRs within the architecture itself. This raises the question of whether RobustFilter is optimal to use within Cronus. That's why in this paper, we wanted to understand whether using other AGRs to distill public labels would work better for reducing model error.

4 Proposed Experiment

We decided to experiment with using other AGR schemes for model distillation, borrowing from the AGRs used in the Cronus paper itself for federated learning (applying them to label aggregation in Cronus as opposed to parameter aggregation in federated learning), doing additional research outside of the paper on possibly implementations of AGRs as well. We had to apply these AGR implementations against different model attacks to see if they actually improved on the original RobustFilter aggregation implementation in any way too. We decided to use the LIE attack and label flip attack as described in section 2.3 and 4.2.

4.1 Alternative AGRs

We will be experimenting with two new AGRs applied to Cronus, as well as combinations of them. We also tried a Bayesian aggregation approach, but scrapped it after realizing that it would not be properly implemented in cronus.

Trimmed Mean. Inspired by the comparison of Federated Learning trimmed mean with the Cronus RobustFilter, we decided to apply a Coordinate-Wise Trimmed Mean to Cronus itself. The only real difference from Federated Learning was that instead of using parameters for the trimming, the logits themselves were used. The way it works is essentially that you take the input vectors of a logit sample from different models, sort per each dimension, and then trim any extremes. The extremes are determined by a preset beta value that indicates what percentage of the highest and lowest values will be trimmed, and we set it to 0.2. We implemented directly following the definition described in paper [5].

Geometric Median. Described in [3], we implemented Weiszfeld's algorithm, an algorithm for computing the geometric median for a data vector. The algorithm, meant to minimize the sum of the l2 distances, is defined in the paper [3] as:

$$\mu^* = \arg\min_{\mu} \sum_{i=1}^{K} \|Y_i - \mu\|_2$$

The algorithm essentially initializes an initial mean, computes vector differences between each model vector for the sample and the mean as well as the euclidean distances, weights these distances, and computes a weighted average, doing this for multiple iterations until convergence.

Bayesian Approach (Scrapped). To make things more interesting, we decided to also try a Bayesian approach for our logit-based aggregation. We wanted to implement the BiLA-CM algorithm as described in [4], which is a label aggregation framework that utilizes a neural network with a softmax activation to predict an unknown true label and noise parameters through Bayesian inference. The algorithm is shown in detail as Algorithm 1 in [4], and was implemented as an object that had continuously updated parameters for each label. However, it turned out to not exactly work properly for our use case, creating an exceptionally high error rate for our models for reasons we don't know why. For this reason, we decided to exclude BiLA-CM from our evaluation, but still believe it could possibly be implemented in a better way to work properly.

Combinational Approach. We also decided to use a combination between our proposed AGRs and the original RobustFilter, seeing if a combination of the directional-based filtering could be combined with our new aggregation schemes to produce an even better result.

4.2 Attack Implementations

To test our different AGRs in Cronus, we decided to test against different poisoning attacks. We decided to use two different attacks that were on opposite ends of the poisoning attack spectrum. As explained in 2.3, we will be using the LIE attack, for stealthy variance, and Label flip attack, for broad sweeping poisoning, in our experiments to test the effectiveness of different Cronus AGRs against

$$s = num\ malicious\ clients$$
$$n = num\ clients$$
$$z_{\max} = \phi^{-1}\left(1 - \frac{s}{n}\right)$$
$$v_{mal} = \mu - z_{\max} \cdot \sigma$$

the original RobustFilter implementation. We will go more in depth into our implementation of said attacks here.

Little is Enough (LIE). A little is Enough attack model creates predictions not based on input, but on a calculated benign mean. This mean is initialized in the first couple of rounds, where the LIE models act completely benign. Then based on the number of collaborating malicious clients, it adds z_max to the benign mean on each update [2].

z_max can be interpreted as the number of additional clients needed to set the median. After many rounds this update slowly poisons the accuracy of the federated learning environment.

We implemented the LIE attack with 5 attack models in an environment of 20 models total.

Label Flip. The Label Flip attack is much simpler which makes it easier to filter in certain cases. The Label Flip simply changes the way that the malicious model perceives its own data. The model architecture can be defined in the exact same way as a benign model, but it learns that every input should be classified as the target class. Over time this will slowly bias the public labels towards the target class as well, especially when the Label Flip clients are close to a majority.

We implemented the label flip attack with 5 attacking models in an environment of 20 models total.

4.3 Implementation of Architecture

We implemented the Cronus architecture for our experiment within a single script. For our client models, we decided to use a standard 3-layer Convolutional Neural Network (layers of size 784, 256, 64) implemented in pytorch, training on the MNIST dataset as described in the Cronus paper. The client models were reserved 50,000 images from the MNIST training dataset, with the remaining 10,000 training images being set aside for the public dataset. The 50,000 images were split up equally amongst clients to create their local datasets. We had an initialization phase for the client models, where they trained with their local dataset. After that, there was a collaboration phase where clients sent their predicted labels for the 10,000 public images to the "server", which then used an AGR defined by functions to aggregate the logits for each sample into soft labels. These aggregated logits were then given to the clients, which then trained on their private dataset for one epoch using Cross Entropy Loss and for another epoch on the aggregated public dataset using Soft KL div loss. We also recorded the error rate of the public dataset as well as each client model on the MNIST test dataset for our experimental results.

Our experiment used 20 models, with 5 models being malicious for the LIE and label flip attacks. We also had 50 initialization epochs and 50 collaboration epochs to match the original Cronus paper.

5 Related Work

In our examination, we did not find many papers relating to the Cronus architecture at all. It seems our examination relating to testing different AGRs is an original idea to be explored. The original Cronus paper had described its comparisons of the architecture solely with other federated learning algorithms, so exploring deeper into Cronus with alternative AGRs to RobustFilter is a good starting point for seeking improvements to the model.

6 Challenges

Implementation of the Cronus architecture was the most challenging aspect of the experimental process. Although the Cronus algorithm was provided, due to inexperience with machine learning notation and conventions, we found difficulty in implementing the code. However, through trial and error with different libraries and methods, we eventually were able to.

Initially, we tried using the flower library, which is a python library that allows efficient implementation of collaborative learning models, typically federated learning, but it can be modified to send aggregated label data as well. However, we found that the library was too unnecessarily complex to work with, so we switched to implementing a client-server architecture ourselves.

We implemented a client-server architecture using sockets in python, with a server script initializing, then receiving and sending aggregated logit values, and the client script initializing and sending model predictions on the public dataset. This was a hassle to work with though, and we decided to go with a simpler scheme.

Finally, we decided to implement the architecture all in one script, With the data of the clients and server being initialized, distributed, then the initialization phase and collaboration phase being done all at once without separation of the server and clients, as described in 4.3. This method was much easier to work with, and was what we decided to go for.

Another area of struggle was implementing the AGRs. After writing the Cronus algorithm itself, we had to pull from the Cronus script for the original RobustFilter AGR, as well as pull from other papers for the AGRs we ended up using. Implementing the algorithms in python was slightly challenging, but through continuously referencing the materials used, we were able to eventually understand and implement the methods.

7 Experiments and Results

7.1 No Attack

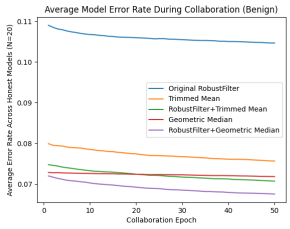

Figure 1: Model Error with No Attack

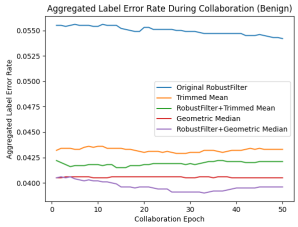

Figure 2: Aggregated Label Error with No Attack

During collaboration, the error rate across the non-RobustFilter aggregation schemes heavily outperform the RobustFilter scheme. Even the starting error rate after the first collaboration epoch is better, suggesting that these other AGRs are much better than RobustFilter. The best performer is the combination of RobustFilter and Geometric Median, which converge to an error rate of below 7%.

7.2 LIE Attack

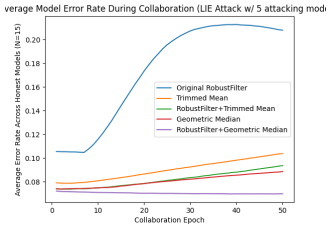

Figure 3: Model Error with LIE Attack

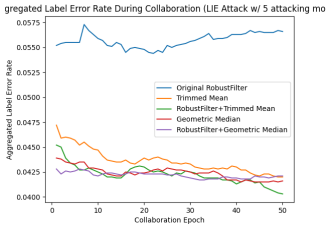

Figure 4: Aggregated Label Error with LIE Attack

RobustFilter is heavily affected by the LIE attack, increasing its error rate to over 20% by the 50th epoch. Meanwhile, the other AGRs don't go above 10%, suggesting much better performance. Again, RobustFilter+Geometric Median perform the best, never even increasing in error and in fact decreased, converging to below 8%.

7.3 Label Flip

The label flip attack still shows RobustFilter as doing the worst and the other AGRs performing better, with RobustFilter again winning out. However, Trimmed Mean seems to increase in error

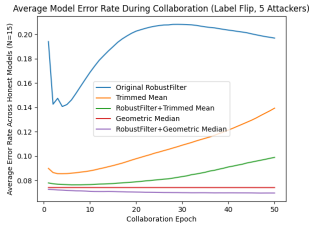

Figure 5: Model Error with Label Flip Attack

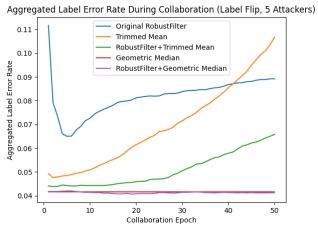

Figure 6: Aggregated Label Error with Label Flip Attack

rate much faster, suggesting that it possibly could not be optimal versus RobustFilter after a certain number of epochs.

Table 1: Aggregated Label Error Under Different Attacks

	AGR Type			
Attack Type	**RobustFilter**	**Trimmed Mean**	**RobustFilter+Trimmed Mean**	**Geometric Median**
No Attack	0.0542	0.0433	0.0421	0.0405
LIE	0.0566	0.0420	0.0403	0.0416
Label Flip	0.0892	0.1068	0.0658	0.0416

Table 2: Average Model Error Across Honest Clients

	AGR Type			
Attack Type	**RobustFilter**	**Trimmed Mean**	**RobustFilter+Trimmed Mean**	**Geometric Median**
No Attack	0.1047	0.07567	0.07073	0.07184
LIE	0.2080	0.1037	0.09359	0.08851
Label Flip	0.1970	0.1394	0.09897	0.07421

8 Conclusion

From our experiments, we have determined that RobustFilter is not an optimal AGR for the Cronus implementation in our experimental scheme. Using Trimmed Mean, Geometric Median, and a combination of the two with RobustFilter yielded much better results than the RobustFilter scheme. Additional experimentation must be done with other datasets and models to verify the validity of our conclusion, however.

References

[1] Moran Baruch, Gilad Baruch, and Yoav Goldberg. 2019. A Little Is Enough: Circumventing Defenses For Distributed Learning. arXiv:1902.06156 [cs.LG] https://arxiv.org/abs/1902.06156?

[2] Hongyan Chang, Virat Shejwalkar, Reza Shokri, and Amir Houmansadr. 2019. Cronus: Robust and Heterogeneous Collaborative Learning with Black-Box Knowledge Transfer. arXiv:1912.11279 [stat.ML] https://arxiv.org/abs/1912.11279

[3] El-Mahdi El-Mhamdi, Sadegh Farhadkhani, Rachid Guerraoui, and Lê-Nguyên Hoang. 2023. On the Strategyproofness of the Geometric Median. In *Proceedings of The 26th International Conference on Artificial Intelligence and Statistics (Proceedings of Machine Learning Research, Vol. 206)*, Francisco Ruiz, Jennifer Dy, and Jan-Willem van de Meent (Eds.). PMLR, 2603–2640. https://proceedings.mlr.press/v206/el-mhamdi23a.html

[4] Chi Hong, Amirmasoud Ghiassi, Yichi Zhou, Robert Birke, and Lydia Y. Chen. 2020. Online Label Aggregation: A Variational Bayesian Approach. arXiv:1807.07291 [cs.LG] https://arxiv.org/abs/1807.07291

[5] Dong Yin, Yudong Chen, Kannan Ramchandran, and Peter Bartlett. 2021. Byzantine-Robust Distributed Learning: Towards Optimal Statistical Rates. arXiv:1803.01498 [cs.LG] https://arxiv.org/abs/1803.01498

9 Acknowledgments

Paper written as a result of research project, assistance from TA Shayan Nazeer and Professor Taqi Raza.